

CENG 499 - Special Topics: Introduction to Machine Learning

Homework 1 - Report

Student: İpek Güren

Student ID: 2310118

Getting Started

You are going to do a sanity check before training the network. Compute the accuracy and loss you expect on the test set. Then, compute the real accuracy and loss on the test with an untrained ANN. Are they similar or not? Write the results you got and comment on them in your report.

The cross-entropy between two probability distributions, such as Q from P, can be stated formally as: $H(P, Q)$. Where $H()$ is the cross-entropy function, P may be the target distribution and Q is the approximation of the target distribution.

Cross-entropy can be calculated using the probabilities of the events from P and Q, as follows:

$$H(P, Q) = - \sum_{x \in X} P(x) * \log(Q(x))$$

Where $P(x)$ is the probability of the event x in P, $Q(x)$ is the probability of event x in Q and log is the base-2 logarithm, meaning that the results are in bits.

I took each $P(X)$ as 0.1 since there are 10 classes and the chance of guessing correctly with random weights is 0.1. Then For the result of softmax function, we know that the output vector of the softmax function is composed of values between 0 and 1. I used the assumption that $Q(x)$, which is the probability of event x in Q, would also be 0.1 in our formula.

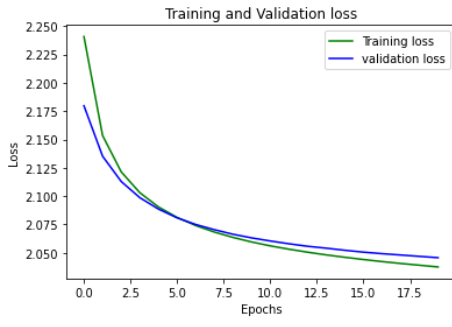
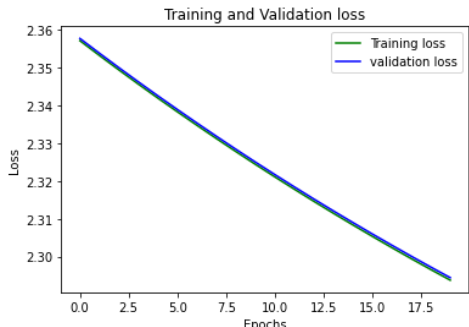
So, the formula looks like:

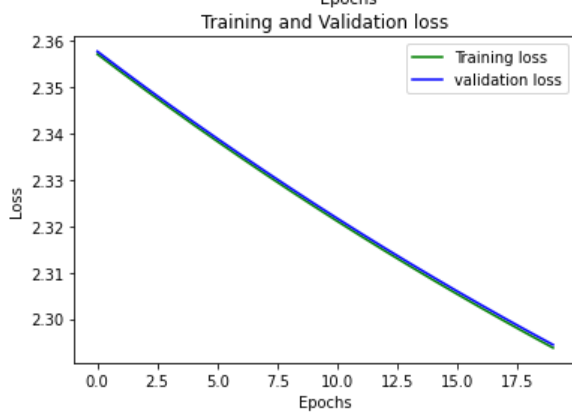
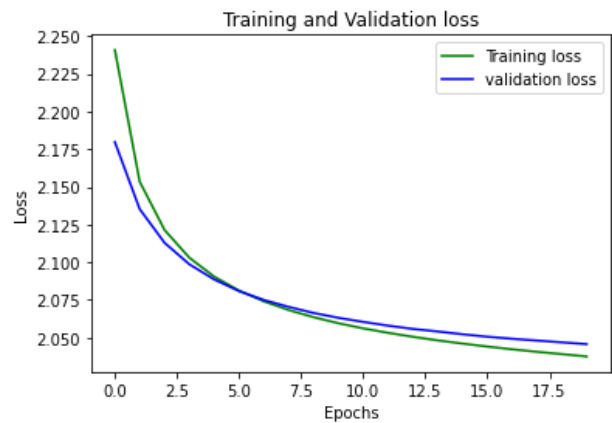
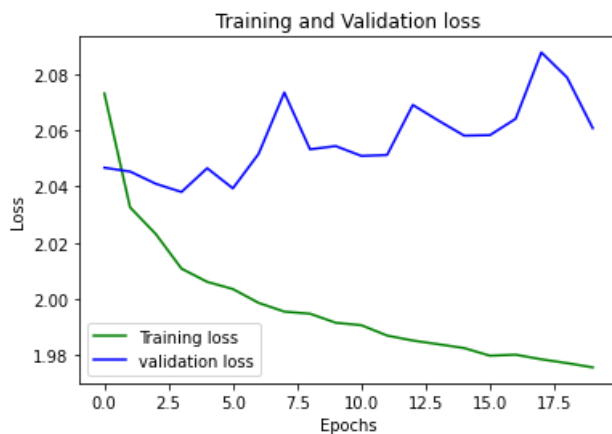
$H(P, Q) = - \sum (0.1) * \log(0.1) = - 10 * (0.1) * \log(0.1) = - \log(0.1) = 3.322 \rightarrow$ predicted loss
This would correspond to the accuracy <10%, around 9% .

When I tested an untrained 3 layered model, the result was: `Test Accuracy = 8.770%`

Hyper Parameter Optimization

1 layer Experiment Results

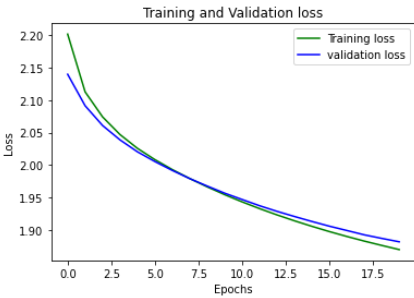
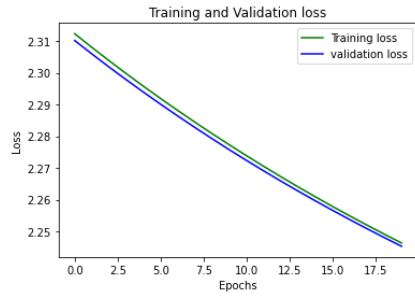
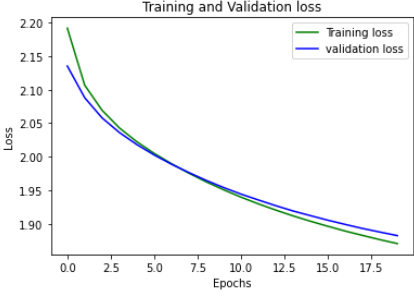
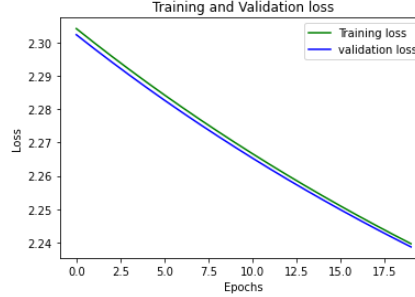
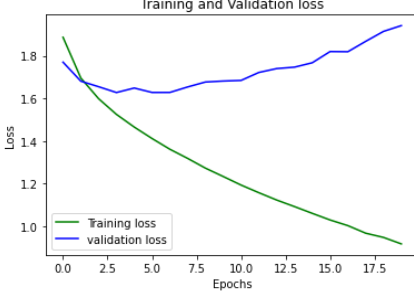
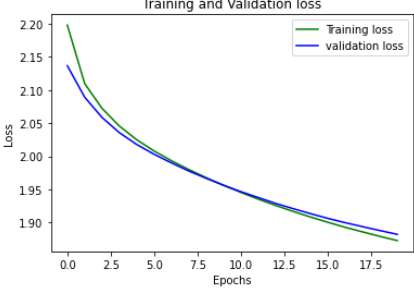
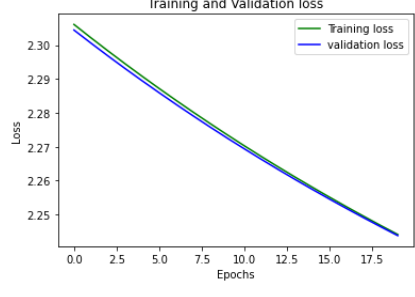
lr = 1e-3	lr = 1e-5	lr = 1e-7
Test acc. = 28.100% Train loss = 1.9761 Val. loss = 2.0676 Val acc. = 5.4783	Test acc. = 30.090% Train loss = 2.0375 Val. loss = 2.0457 Val acc. = 5.6054	Test acc. = 12.850% Train loss = 2.2937 Val. loss = 2.2944 Val acc. = 2.3954
		



(Graphs printed out for better view.)

Best Resulting Configuration was with learning rate 1e-5, as we can see from the graph, validation loss and training loss are close to each other and decreasing together.

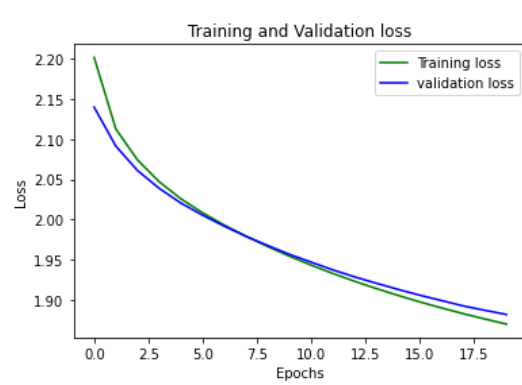
2 layer Experiment Results

	lr = 1e-3	lr = 1e-5	lr = 1e-7
f = "hardswish"	Test acc. = 42.350% Train loss = 0.9177 Val. loss = 1.9690 Val acc. = 8.6988 	Test acc. = 34.380% Train loss = 1.8698 Val. loss = 1.8818 Val acc. = 6.4482 	Test acc. = 19.960% Train loss = 2.2463 Val. loss = 2.2453 Val acc. = 3.1503 
f = "relu"	Test acc. = 42.910% Train loss = 0.9109 Val. loss = 1.9350 Val acc. = 8.6675 	Test acc. = 34.300% Train loss = 1.8703 Val. loss = 1.8822 Val acc. = 6.4320 	Test acc. = 19.830% Train loss = 2.2396 Val. loss = 2.2386 Val acc. = 3.9240 
f = "tanh"	Test acc. = 42.720% Train loss = 0.9175 Val. loss = 1.9416 Val acc. = 8.6338 	Test acc. = 34.660% Train loss = 1.8729 Val. loss = 1.8824 Val acc. = 6.4268 	Test acc. = 19.340% Train loss = 2.2441 Val. loss = 2.2437 Val acc. = 3.7964 

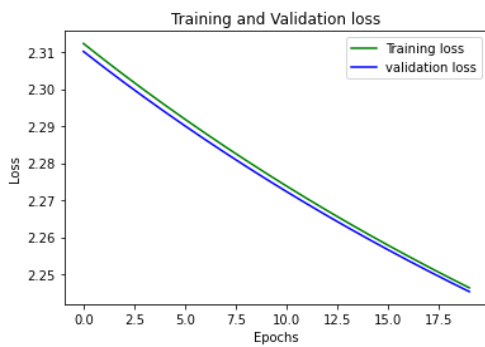
f = "hardswish", lr = 1e-3



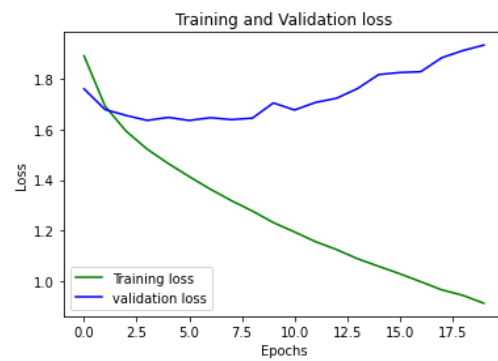
f = "hardswish", lr = 1e-5



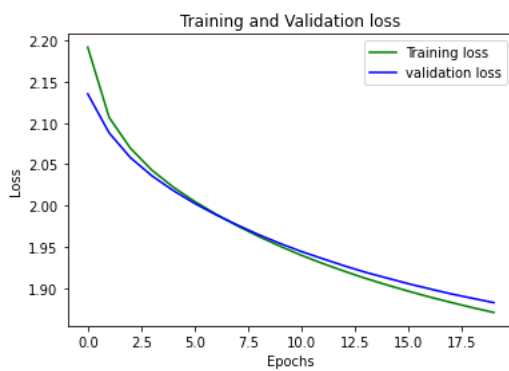
f = "hardswish", lr = 1e-7



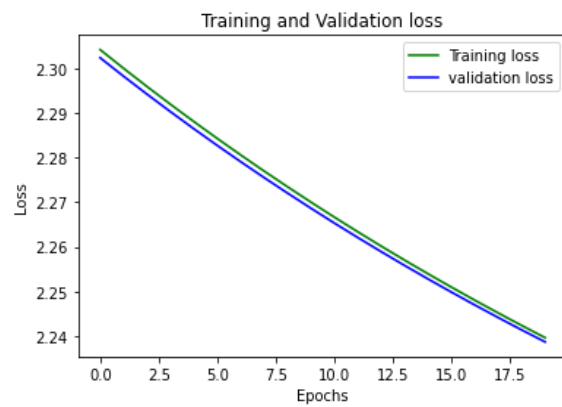
f = "relu", lr = 1e-3



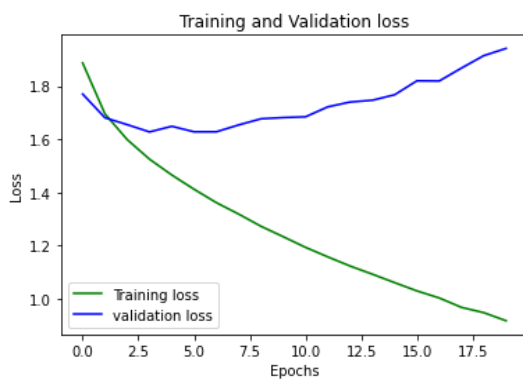
f = "relu", lr = 1e-5



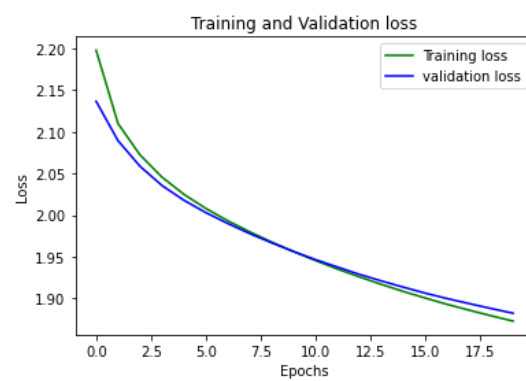
f = "relu", lr = 1e-7



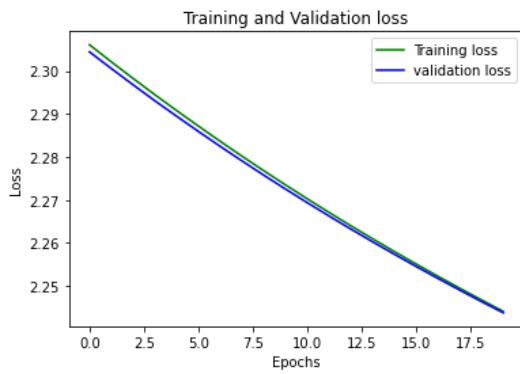
f = "tanh", lr = 1e-3



f = "tanh", lr = 1e-5

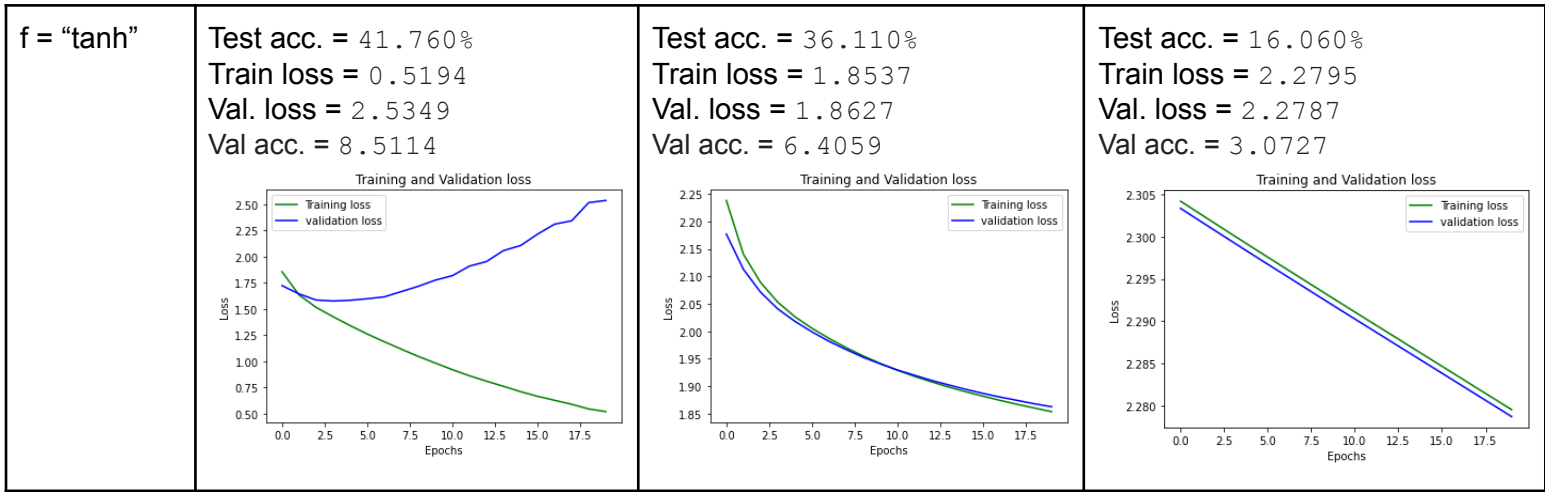


f = "tanh", lr = 1e-7

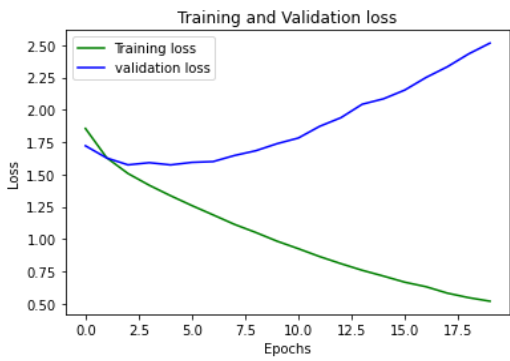


3 layer Experiment Results

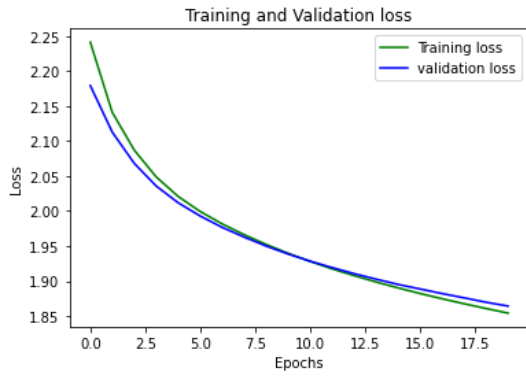
	lr = 1e-3	lr = 1e-5	lr = 1e-7
f = "hardswish"	Test acc. = 42.910% Train loss = 0.5204 Val. loss = 2.5160 Val acc. = 8.6391	Test acc. = 39.090% Train loss = 1.8542 Val. loss = 1.8642 Val acc. = 6.4347	Test acc. = 18.060% Train loss = 2.2778 Val. loss = 2.2775 Val acc. = 3.5690
f = "relu"	Test acc. = 43.380% Train loss = 0.5157 Val. loss = 2.5814 Val acc. = 8.5928	Test acc. = 37.070% Train loss = 1.8481 Val. loss = 1.8563 Val acc. = 6.4688	Test acc. = 19.800% Train loss = 2.2744 Val. loss = 2.2742 Val acc. = 3.5556



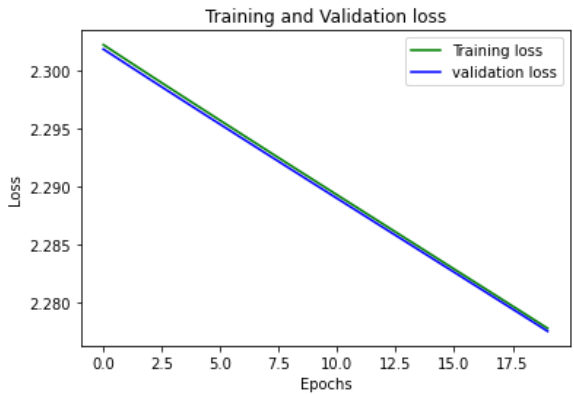
f = "hardswish", lr = 1e-3



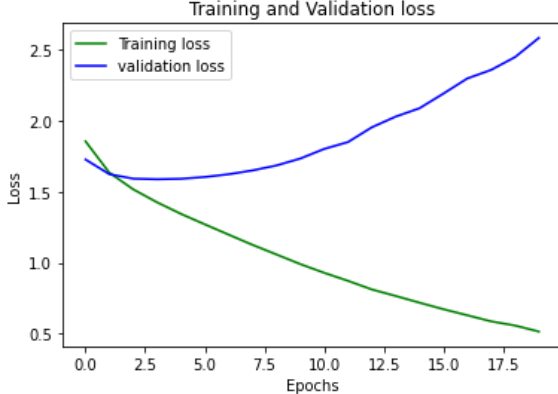
f = "hardswish", lr = 1e-5



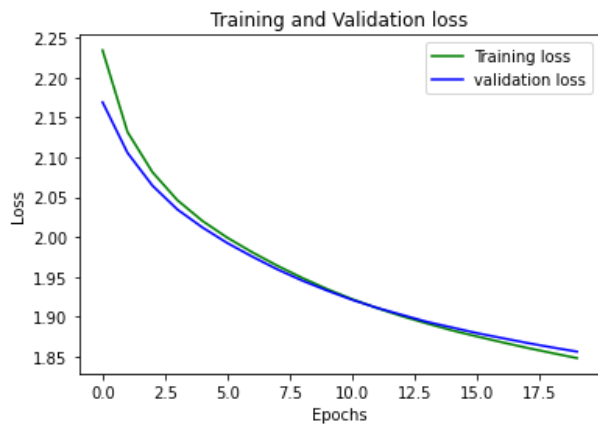
f = "hardswish", lr = 1e-7



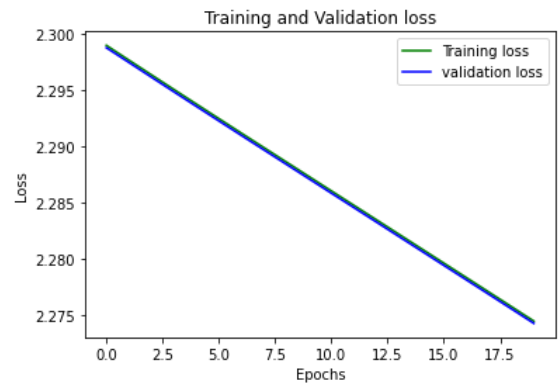
f = "relu", lr = 1e-3



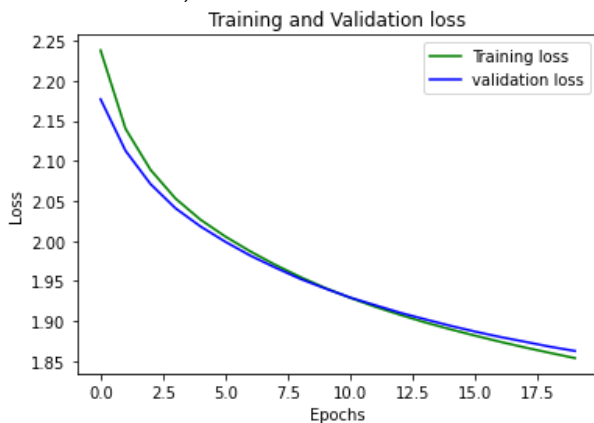
f = "relu", lr = 1e-5



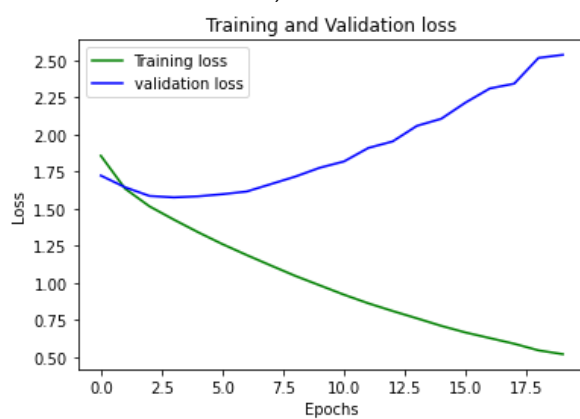
f = "relu", lr = 1e-7



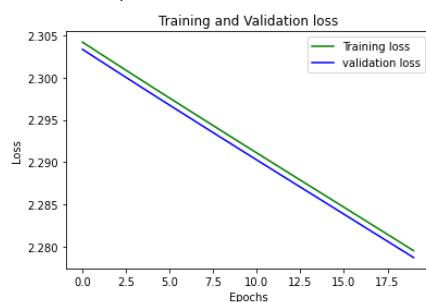
f = "tanh", lr = 1e-3



f = "tanh", lr = 1e-5



f = "tanh", lr = 1e-7



Some Comments on Results:

- The best performing configurations were:
 - Layer 1: 30.090% test accuracy with learning rate = 1e-5.
 - Layer 2: 42.910% test accuracy with learning rate = 1e-3, and activation function = "relu". This is interesting since the epoch number was the same with 1 Layered model, but this time, a bigger step size was necessary to meet our minimum.
 - Layer 3: 43.380% test accuracy with learning rate = 1e-3, and activation function = "relu". Which is also interesting since I expected a much larger accuracy when compared to the 2 Layered model, however it received a slight increase. However, when I applied L2 regularization the test accuracy was 45.010%, which proves itself to be a useful method of optimization.

What countermeasures did you take against overfitting?

- I decided on regularization. L1, L2, and drop-down methods are tried here:
- I tried the L2 method. It is used for reducing the overfitting if it is occurring. L2 regularization acts like a force that removes a small percentage of weights at each iteration. So, the model can learn from slightly different networks at each pass.
- The most popular regularization terms of this kind are L2 regularization, which is the sum of squares of all weights in the model, and L1 regularization, which is the sum of the absolute values of all weights in the model. ()

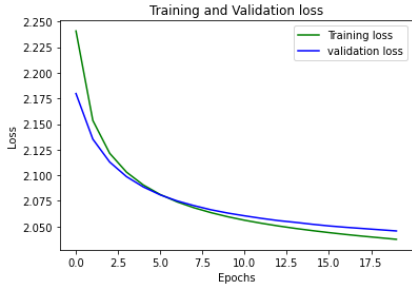
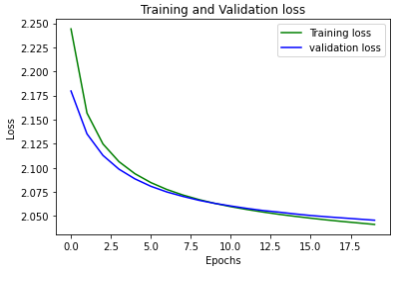
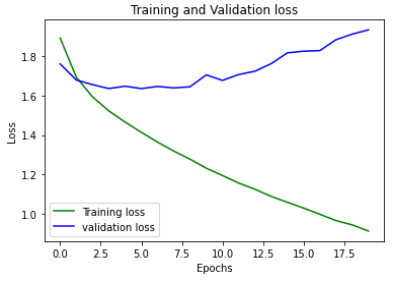
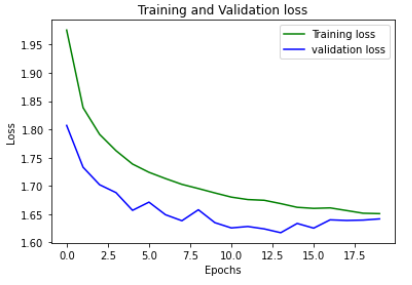
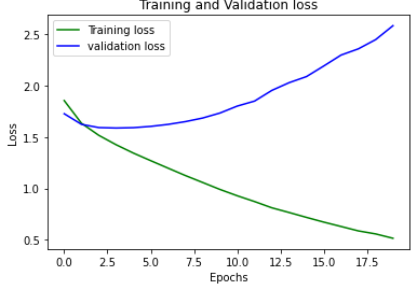
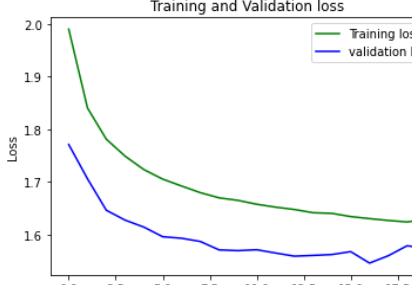
- The code is this part:

```
l2_lambda = 0.001
l2_norm = sum(p.pow(2.0).sum()
               for p in model.parameters())
loss = loss + l2_lambda * l2_norm
```

Replaces pow(2.0)
with abs() for L1
regularization

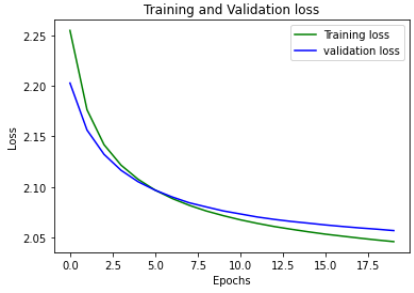
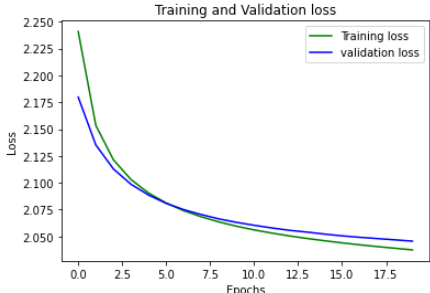
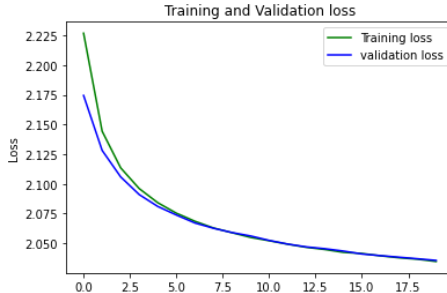
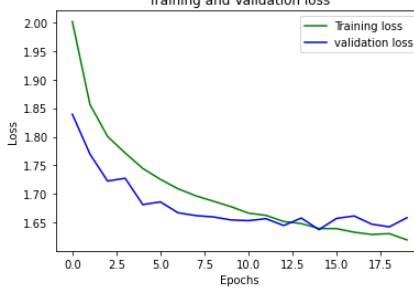
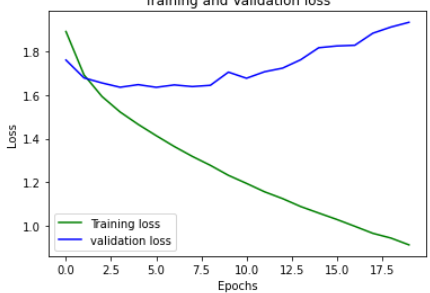
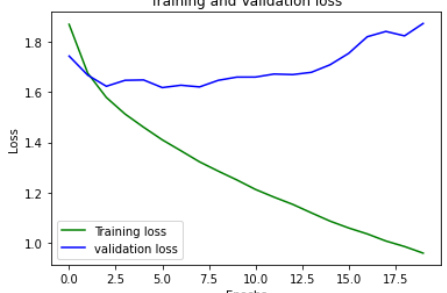
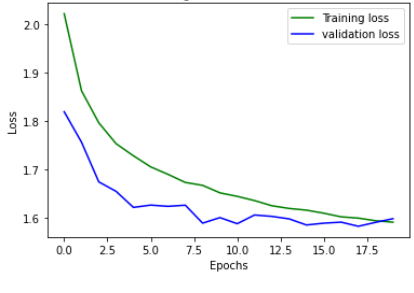
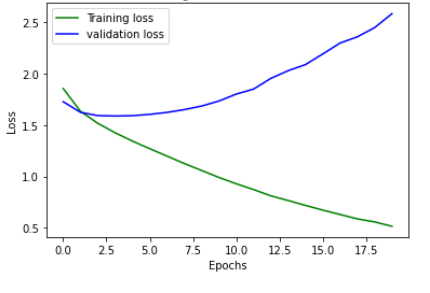
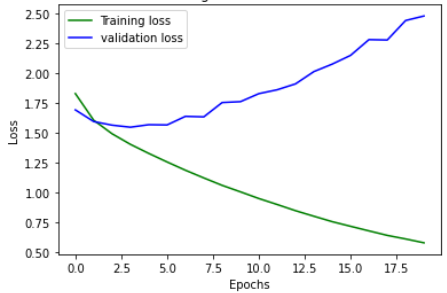
(code obtained from [Deep Learning with PyTorch book by Eli Stevens, Luca Antiga, Thomas Viehmann](#))

- After I added this method, I tried it on the 3 best scoring configurations for each layer.

	Test accuracy before L2	Test accuracy after L2
layer = 1	30.090% 	30.120% 
layer = 2	42.910% 	43.240% 
layer = 3	43.380% 	45.010% 

As we can see, L2 regularization slightly improved our results, however it was not as drastic of a change as I expected, making me believe that maybe overfitting was not a major issue in this case.

I also tried different partitions on the training set and validation set on the best performing configurations of each different layered model. The results are in the following table.

	60% train, 40% val.	80% train, 20% val.	95% train, 5% val.
layer = 1	test acc: 28.360% 	test acc: 30.090% 	test acc: 29.090% 
layer = 2	test acc: 42.890% 	test acc: 42.910% 	test acc: 43.400% 
layer = 3	test acc: 44.420% 	test acc: 43.380% 	test acc: 42.220% 

As we can observe from the table, with partitions 60% train, 40% val. and 95% train, 5% val., the test accuracies are slightly lower. An interesting part is how 2 layered model performed better than 3 layered model when the training set was overwhelmingly larger than the validation set.

How may one understand when a network starts to overfit during training?

- I think we can say that a rule of thumb definition is that overfitting occurs when the train accuracy keeps improving while your validation accuracy stops improving

(or even starts getting worse). In this case, it may be the best option to stop training when the results stop improving, before they take a downturn. I decided to check different epoch counts in the range [10,40]. I believe that around 20 epochs performs best for my models, so I used this number for all my experiments.

What method did you use to decide where to end the training procedure?

- I observed the graphs and numerical results for each epoch number, and when I realized there was a point where my results got worse, I stopped training the model.

Discuss whether the accuracy is a suitable metric to measure the performance of a network for this specific dataset.

- I believe that Logarithmic Loss would be very fitting to this type of task, since Logarithmic Loss works by penalising the false classifications. It seems to be working well for multi-class classification. On one source it is stated that when working with Log Loss, the classifier must assign probability to each class for all the samples, but maybe this would cause the model to take more time to perform train and test operations. However, since our dataset is small enough and we are utilizing fairly minimal operations, Logarithmic Loss seems to be a good metric to use in this case.
- Another option could be F1-scores, since it is very balanced and paints the picture very clearly, since it is a harmonic mean between precision and recall. It can state the fragile trade-off between these two metrics. I believe many people resort to F1-scores when their system is rather unreliable or, rather, easy to measure incorrectly.

Learning Rates:

- Let us examine this table obtained by [this article](#) (Wilson,2011)

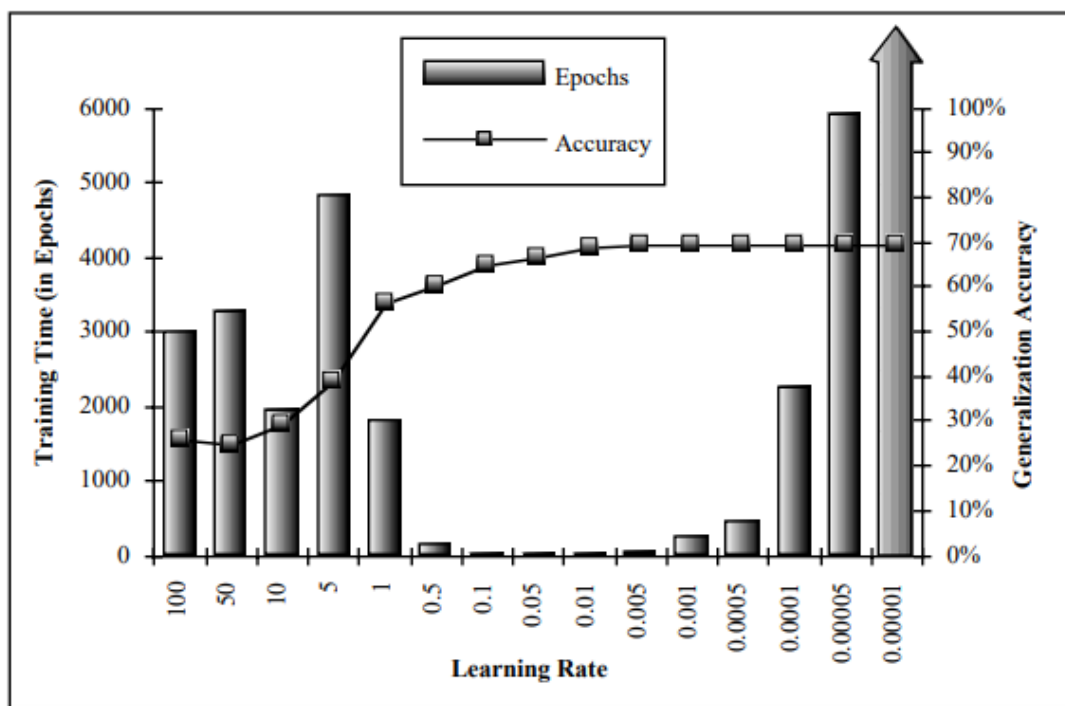


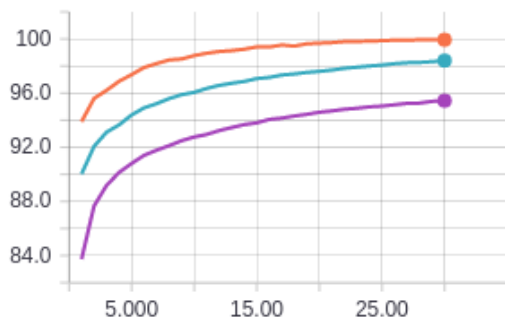
Figure 1: Training time (in epochs) and maximum hold-out set phoneme generalization accuracy for each learning rate, on a digit speech recognition task. The bars indicate the number of epochs needed to reach the maximum generalization accuracy, and the line indicates what the maximum accuracy was for each learning rate.

- When the epoch count is the largest, the smallest learning rate is used to obtain high accuracy. I visualize both epochs and learning rates in my mind as walking. If I was walking a 100m distance, I could increase my step count(epochs) and decrease my step distance(learning rate). If there was a coin on this road(our global minimum), I would have a higher chance of stepping on it this way. On the other hand, I also could decrease my step count(epochs) and increase my step distance(learning rate). This way, I could go much faster but I would probably not step on the coin. From the table, we can see that the number of epochs is much smaller when the learning rate is as large as 0.5, as opposed to the 0.00005 with a huge number of epochs. However, we can also see the time it took the developers to train their model with the given amount of epochs, and the model with rate=0.00005 took the longest time to run. This is a major disadvantage of the small learning rates. On the other hand, we can see how the accuracy line is at its lowest in the learning rate in range [100, 0.5]. They were able to achieve only 20%~ accuracy with their models, but it surely was achieved much faster than models with smaller learning rates. We can say that accuracy and speed are the trade-offs when deciding on a fixed learning rate.

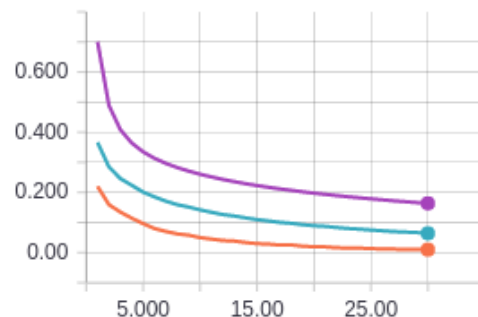
Batch size:

Let us take a look at these graphs I obtained from [this source](#):

train accuracy

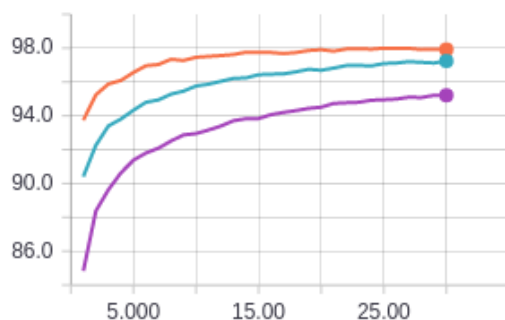


train loss

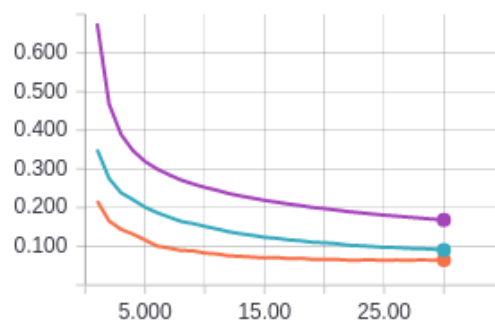


Training loss and accuracy when the model is trained using different batch sizes.

test accuracy



test loss



Testing loss and accuracy when the model is trained using different batch sizes.

These graphs belong to a neural network trained with batch sizes on MNIST dataset:

- orange: 64
- blue: 256
- purple: 1024

As we can observe, for this dataset(which is kind of similar to CIFAR set as well), 64 and 256 batch sizes reduce the loss and increase the accuracy more than 1024 batch size. The blue line starts higher than the orange line, but meets the orange line in terms of performance.

It has been observed in practice that when using a larger batch there is a significant degradation in the quality of the model, as measured by its ability to generalize. (From Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, Ping Tak Peter Tang. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. <https://arxiv.org/abs/1609.04836>)

So, we can say that overly increasing the batch size is detrimental to the training of an ANN.

When you put m examples in a minibatch, you need to do $O(m)$ computation and use $O(m)$ memory, but you reduce the amount of uncertainty in the gradient by a factor of only $O(\sqrt{m})$. In other words, there are diminishing marginal returns to putting more examples in the minibatch. (From [The Deep Learning Book](#))

However, the batch size is too small, so we are not considering enough input at once. Small batches can offer a regularizing effect (Wilson and Martinez, 2003) due to the noise they add to the learning process. Generalization error is often the highest for a batch size of 1. Training with such a small batch size might require a small learning rate to maintain stability because of the high variance in the estimate of the gradient. The total runtime can be very high as a result of the need to make more steps, both because of the reduced learning rate and because it takes more steps to observe the entire training set.